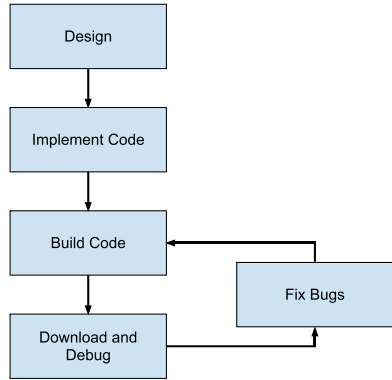


Overview and SDLC

Developing for an embedded system is an entirely different ball game as compared to a traditional non-embedded system. This is due to the constraints imposed by these systems that we mentioned in the previous chapter - hardware, resources and time. A program has to be usually customised for the platform that has to be deployed on i.e. cross-compiled for each of



the platforms you want it to work on. Another thing to consider is that you'll also have very limited functionality in terms of what one refers to as a standard library, only the most important functions find their way there. There may also be a limitation in the memory which affects the size of program that you can deploy to most systems.

And of course, time, the ever present elephant in the room – some tasks will have a hard-deadline i.e. they have to be completed on or before the deadline after that the task has failed even if the result is logically correct. Say for example you're making a smart electricity meter, one with a GSM module baked in on the board which periodically sends the electricity usage and at the same time take and execute actions based on commands from a command and control server. For such a system, the remote command takes the priority above everything else, say for example if the supervisor sends a command to switch off the power supply – the result should be instantaneous, this task takes precedence above every other task that the system is performing. You will have to make design decisions and write your software in such a way that such systems can run reliably, in all conditions, as once they've been deployed it's difficult to make modifications in such systems.

The life-cycle for a typical embedded software looks something like the figure shown below. You start with designing the system at a conceptual level after having collected all your requirements and having clearly defined your project goals. After this you write your code in the language of your choice supported by the platform. This is followed by building your code, this is where the compiler comes in, the compiler(s) that you use for your project will take the code and create an executable which depends on your platform can be pure assembly or optimised bytecode. After this you deploy your